

String RegEx



Practicar

Pots practicar i provar les expressions regulars a les següents pàgines online:

<http://www.regexr.com/>

<https://regex101.com/>

Agrupació de caràcters

L'agrupació de caràcters per formar una expressió regular es fa mitjançant `[]`.

Character Class	Descripció
<code>[abc]</code>	La cadena ha de contenir els caràcters "a" o "b" o "c" (cas simple). Es mira si el caràcter a avaluar és algun de la llista.
<code>[abc][12]</code>	La cadena ha de contenir els caràcter "a", "b", o "c" seguit d'un "1" o un "2". Exemple: a1, a2, b1, b2, c1 o c2
<code>[^abc]</code>	La cadena ha de contenir qualsevol caràcter excepte "a", "b", o "c" (negació). Per exemple: d, e, f, g, h, i, ...
<code>[a-zA-Z]</code>	La cadena ha de contenir un caràcter entre "a" i "z" o "A" i "Z", inclusivus (rang)
<code>[a-d[m-p]]</code>	La cadena ha de contenir un caràcter entre "a" i "d", o entre "m" i "p" = <code>[a-dm-p]</code> (unió)

Character Class	Descripció
[a-z&&[c-o]]	La cadena ha de contenir un caràcter entre "a" i "z" i també entre "c" i "o", per tant ha de contenir un caràcter entre "c" i "o" (intersecció)
[a-z&&[^bc]]	La cadena ha de contenir un caràcter entre "a" i "z", excepte "b" i "c" = [a-d-z] (subtracció)
[a-z&&[^m-p]]	La cadena ha de contenir un caràcter entre "a" i "z", i no entre "m" i "p" = [a-lq-z] (subtracció)
a b	La cadena ha de contenir "a" o "b". Exemple: "Hola Adeu" La cadena ha de contenir "Hola" o "Adeu"
ab	La cadena ha de contenir "a" seguit de "b". Exemple: "HolaAdeu" La cadena ha de contenir "HolaAdeu"
.	La cadena ha de contenir qualsevol caràcter excepte el canvi de línia.
.abc.	La cadena ha de contenir qualsevol caràcter excepte el canvi de línia, seguit de "abc" i un altre caràcter excepte el canvi de línia. Exemple: 1abc3, zabcA.
^abc	La cadena ha de començar per "abc". Exemple: ^abc[0-9] La cadena ha de començar per "abc" seguit d'un nombre entre "0" i "9", per exemple: abc0, abc1, abc2, abc9, ...
abc\$	La cadena ha de finalitzar amb "abc". Exemple: ^abc[0-9][A-Z]\$ La cadena ha de començar per "abc" seguit d'un nombre entre "0" i "9", i acabar amb una lletra entre "A" i "Z", per exemple: abc0A, abc1C, abc2D, abc9Z, ...

Altres exemples:

- [a-zA-Z] La cadena pot contenir qualsevol lletra minúscula (a-z) o majúscula (A-Z).
- [a-z&&[^m-p]] La cadena pot contenir qualsevol lletra minúscula menys qualsevol lletra

minúscula entre m i p incloses.

- `[a-z1-9]` La cadena pot contenir qualsevol lletra minúscula (a-z) i/o qualsevol dígit de l'1 al 9 (ambdós inclosos).

Accents:

- `[A-zÀ-ú]` Accepts lowercase and uppercase characters
- `[A-zÄ-ÿ]` As above but including letters with an umlaut (includes `^ \ × ÷`)
- `[A-Za-zÄ-ÿ]` As above but not including `^ \`
- `[A-Za-zÄ-Öø-öÿ]` As above but not including `^ \ × ÷`

Metacaràcters

- `\d` Indica un dígit. Equival a `[0-9]`.
- `\D` Indica un no dígit. Equival a `^[0-9]` o `^[^d]`
- `\s` Indica un espai en blanc. Equival a `[\t\n\x0b\r\f]` (Compte, hi ha un espai abans de `\t`)
- `\S` Indica un no espai en blanc. Equival a `^[^s]`
- `\w` Indica una lletra majúscula o minúscula, un dígit o el guio baix. Equival a `[a-zA-z0-9_]`
- `\W` Equival a `^[^w]`

Utilitzar dues barres inclinades per mostrar qualsevol dels metacaràcters:

- `\.` Indica un punt
- `\^` Indica el caràcter `^`

Quantificadors

Hi ha un conjunt de símbols especials que ens permeten controlar la repetició dels caràcters.

Regex	Description
<code>X?</code>	X es repeteix de 0 a 1 vegada. Exemple: <code>[a-z]?</code> La cadena conte una lletra minúscula o no en conté cap.
<code>X+</code>	X es repeteix de 1 a més vegades. Exemple: <code>Ho+la</code> La cadena ha de començar per "H" finalitzar per "la", i entre "H" i "la" poden haver d'1 a N "o". Un exemple de cadena que compliria l'expressió seria "Hola" o "Hoooooola".

Regex	Description
<code>X*</code>	X es repeteix de 0 a més vegades. Exemple: <code>[abc]*</code> Els caràcters "a" o "b" o "c" i s'han de repetir de 0 a N vegades. Un exemple de cadena que compliria l'expressió seria "aaa", "abcabcabc", "acbaaccab", ...
<code>X{n}</code>	X es repeteix n vegades. Exemple: <code>[a-z&&[^bc]]{6}</code> La cadena ha de contenir 6 lletres minúscules excepte la "b" o la "c".
<code>X{n,}</code>	X es repeteix n o més vegades. Exemple: <code>[a-zA-Z]{2,}</code> La cadena ha de contenir com a mínim 2 lletres minúscules o majúscules.
<code>X{y,z}</code>	X es repeteix de y a z vegades. Exemple: <code>[a-zA-Z]{2,6}</code> La cadena ha de contenir com a mínim 2 lletres minúscules o majúscules i com a màxim 6.

Grups

`(...)`

Els parèntesis (grups) es poden utilitzar per separar expressions regulars dins una expressió complexa, tal com faríem amb les expressions aritmètiques. Per exemple:

```
"(Bon dia|Bona tarda|Bona nit):([A-Z]{1,4})"
```

Representa una cadena amb qualsevol de les salutacions "Bon dia" o "Bona tarda" o "Bona nit", seguida de dos punts ":" i seguida d'un conjunt d'entre 1 o 4 lletres majúscules "`[A-Z]{1,4}`".

Cadenes que complirien aquesta expressió: "Bon dia:ABE", "Bona tarda:D" i "Bona nit:RUUD"

`(?i)` Ignora majúscules i minúscules del grup de dades que ve a continuació.

Exemple:

```
(?i)([a-z]{3,6})
```

Busca de 3 a 6 lletres de la "a" a la "z" majúscules o minúscules.

Negative look ahead

```
x(?y)
```

Exclou patrons. Trobarà una coincidència si la cadena conté "x" però després de la "x" no tenim una "y".

Exemple:

```
a(?b)
```

Busca "a" però després no pot haver-hi una "b".

Ús d'expressions regulars

Una expressió regular és una seqüència de caràcters que formen un patró de cerca. Cal importar les següents classes:

```
import java.util.regex.Matcher;  
import java.util.regex.Pattern;
```

Definim un patró

Definim l'expressió regular amb el mètode **compile** de la classe **Pattern**. Tenim dues opcions:

Opció 1: Sense distingir majúscules i minúscules: de 3 a 6 lletres de la "a" a la "z" en minúscules.

```
Pattern pattern = Pattern.compile("[a-z]{3,6}");
```

Opció 2: Distingint majúscules i minúscules: el mateix que abans però en majúscules o minúscules.

```
Pattern pattern = Pattern.compile("[a-z]{3,6}", Pattern.CASE_INSENSITIVE);
```

Text a analitzar

Declarem el text a analitzar amb el mètode `matcher` de la classe `Pattern`, aquest mètode retorna un objecte de la classe `Matcher`.

```
Matcher matcher = pattern.matcher("This is a text to search a pattern  
expression.")
```

Patró trobat

Amb la classe `Matcher` tenim els següents mètodes:

Mètode	Significat
matches	true si la cadena coincideix amb la seva totalitat.
find	true si tenim una altra coincidència (comença des de l'inici de la cadena). Un cop trobada, es poden fer servir els següents mètodes per processar els resultats:
group	ens retorna el text trobat.
start	ens indica la posició on comença el text trobat.
end	ens indica la posició on acaba el text trobat.

Mètode find()

En aquest exemple, comprovem que el codi "adrg" compleix el pattern entre 3 i 6 lletres de la **a** a la **z** en minúscules i majúscules.

El mètode retorna true si el codi compleix el pattern.

```
Pattern pattern = Pattern.compile("[a-z]{3,6}", Pattern.CASE_INSENSITIVE);
Matcher matcher = pattern.matcher("adrg");
System.out.println(matcher.find());
```

Mètodes group(), start() i end()

Trobar la 1a coincidència

Per trobar totes les coincidències dins d'un text:

- Primer comprovem que s'ha trobat una coincidència.
- El mètode group() ens retorna el text trobat.
- El mètode start() indica la posició dins del text.
- El mètode end() indica la posició final en el text.

Pattern a cercar: entre 3 i 6 lletres de la "a" a la "z" seguit d'una "t" en majúscules o minúscules:

```
Pattern pattern = Pattern.compile("[a-z]{3,6}[t]", Pattern.CASE_INSENSITIVE);
Matcher matcher = pattern.matcher("This is a text to search a pattern
duplication.");
if(matcher.find()) {
```

```

        System.out.print("Text: " + matcher.group());
        System.out.print(" (Start: " + matcher.start());
        System.out.print(", End: " + matcher.end()+ ")\n");
    } else {
        System.out.println("Not found!");
    }
}

```

Output:

```
Text: text (Start: 10, End: 14)
```

Exemple de codi: Trobar totes les coincidències

```

Pattern pattern = Pattern.compile("[a-z]{3,6}[t]", Pattern.CASE_INSENSITIVE);
Matcher matcher = pattern.matcher("This is a text to search a pattern
duplication.");

while(matcher.find()) {
    System.out.print("Text: " + matcher.group());
    System.out.print(" (Start: " + matcher.start());
    System.out.print(", End: " + matcher.end()+ ")\n");
}

```

Output:

```
Text: text (Start: 10, End: 14)
Text: patt (Start: 27, End: 31)
Text: uplicat (Start: 36, End: 43)
```

Capturing groups

Els grups es defineixen entre parèntesis, i es poden capturar per ser processats posteriorment, per exemple la següent expressió conté 4 grups:

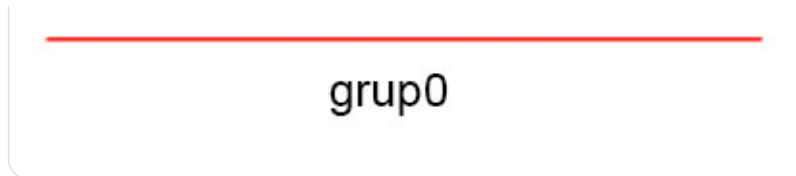
```
((A)(B(C)))
```

Els grups són els següents.

- Grup 0: ((A)(B(C)))
- Grup 1: (A)
- Grup 2: (B(C))
- Grup 3: (C)

[0-9]{2} / [0-9]{2} / [0-9]{4}

grup1 grup2 grup3



En aquest exemple capturem 3 grups, un per processar el dia, un altre pel mes i un altre per l'any.

```
([0-9]{2})/([0-9]{2})/([0-9]{4})
```

grup1 grup2 grup3

- El grup 1 comprova que tenim dos dígets del 0 al 9, per exemple: 01, 02, 12, 35, 55, 99.
- El grup 2 és igual que el grup 1.
- El grup 3 comprova que tenim 4 dígets del 0 al 9, per exemple: 0002, 1800, 2020, 5555, 9999.

```
public class CapturingGroups {
    public static void main(String[] args) {
        checkDateFormat("31/10/2023");
    }

    public static void checkDateFormat(String date) {
        Pattern pattern = Pattern.compile("([0-9]{2})/([0-9]{2})/([0-9]{4})");

        Matcher matcher = pattern.matcher(date);
        if (matcher.matches()) {
            int day = Integer.parseInt(matcher.group(1));
            int month = Integer.parseInt(matcher.group(2));
            int year = Integer.parseInt(matcher.group(3));

            System.out.println("Found group 0: " + matcher.group(0) );
            System.out.println("Found group 1: " + day );
            System.out.println("Found group 2: " + month );
            System.out.println("Found group 3: " + year );
        }
    }
}
```

Output:

```
Found group 0: 31/10/2023
Found group 1: 31
Found group 2: 10
Found group 3: 2023
```

Example Email Verification

Hi ha diferents expressions regulars per comprovar si un text és una direcció d'email. Aquest seria un exemple.

```
"^[\\w-]+(\\. [\\w-]+)*@[A-Za-z0-9]+(\\. [A-Za-z0-9]+)*(\\.[A-Za-z]{2,})$"
```

Expressió	Significat
<code>^[\\w-]+</code>	^ Indica inici de l'email. + Indica que han d'aparèixer 1 o més vegades els caràcters entre claudàtors. \\w Indica caràcters de l'A a la Z tant majúscules com minúscules, dígit del 0 al 9 i el guió baix "-". - També pot incloure el guió "-". En comptes d'utilitzar \\w podem escriure el rang de caràcters amb l'expressió regular: <code>[A-Za-z0-9_-]+</code>
<code>(\\. [\\w-]+)*</code>	A continuació: * Indica que aquest grup pot aparèixer 0 o més vegades. \\. Pot contenir de forma opcional el punt. [\\w-]+ Seguit d'un o més caràcters entre claudàtors, amb el signe + que indica que ha d'aparèixer 1 o més vegades.
<code>@</code>	@ A continuació ha d'incloure el caràcter @
<code>[A-Za-z0-9]+</code>	+ Indica que han d'aparèixer 1 o més vegades els caràcters entre claudàtors. <code>[A-Za-z0-9]</code> Els caràcter poden ser de l'a a la z o de 0 a 9, majúscules o minúscules
<code>(\\. [A-Za-z0-9]+)*</code>	* indica que aquest grup pot aparèixer 0 o més vegades. \\. El grup ha d'incloure un punt. <code>[A-Za-z0-9]</code> El grup ha d'incloure els caràcters de l'a a la z o de 0 a 9, majúscules o minúscules.
<code>(\\. [A-Za-z]{2,})\$</code>	Seguit d'un punt i al menys 2 dels caràcters que apareixen entre claudàtors \$ Indica final de l'email.

Serien vàlids els següents emails:

```
name.surname1234_35AN@gmail.com  
NAME_123_text45AZ@company.co.uk
```

No serien vàlids els següents emails:

```
name.surname?1234@gmail.co.uk  
NAME_123?_text45AZ@company123.co.uk
```

Un altre regex vàlid per comprovar l'email:

```
"^[\\w!#$%&'*/=?`{|}~^-.]+(?:\\.[\\w!#$%&'*/=?`{|}~^-.]+)*@(?:[a-zA-Z0-9-]+\\.  
\\.)+[a-zA-Z]{2,6}$"
```

License and Copyright

Copyright (C) Joan Sèculi (joan.seculi@gmail.com).

Permission is granted to copy, distribute, and/or modify this document under the terms of the **GNU Free Documentation License**, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts.

A copy of the license is available at: <http://www.gnu.org/licenses/fdl.txt>